Fatima Jinnah Women University

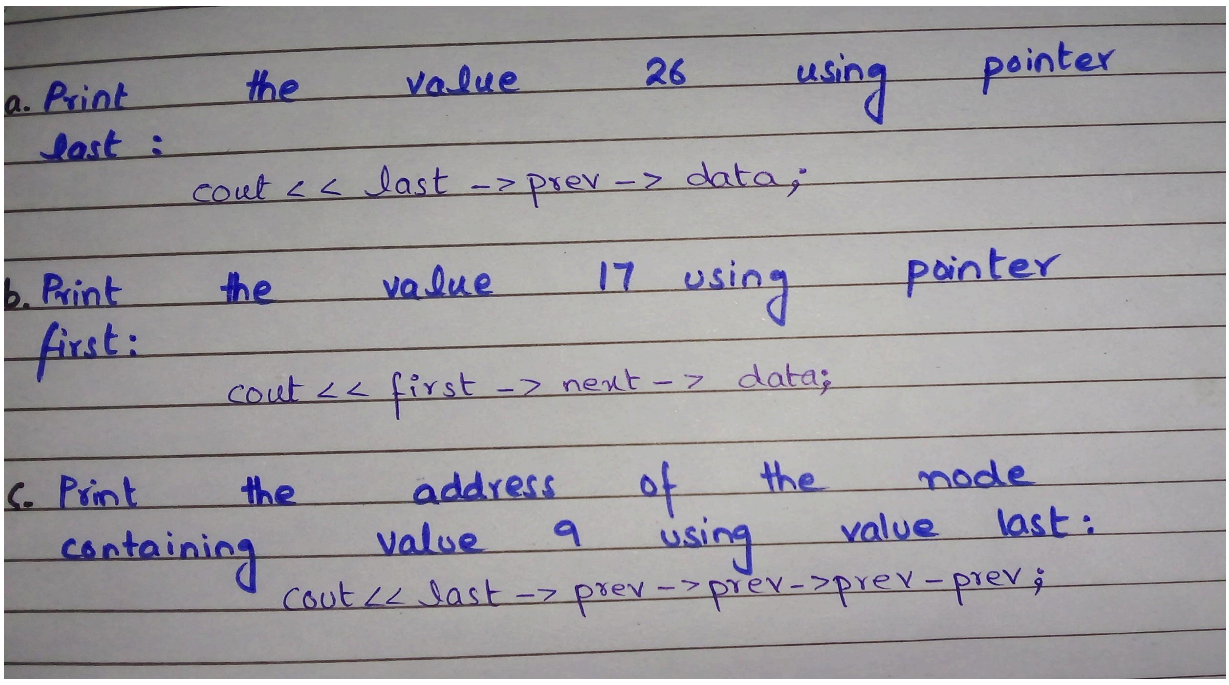
Opening Portals of Excellence Through Higher Education

Data Structures And Algorithms

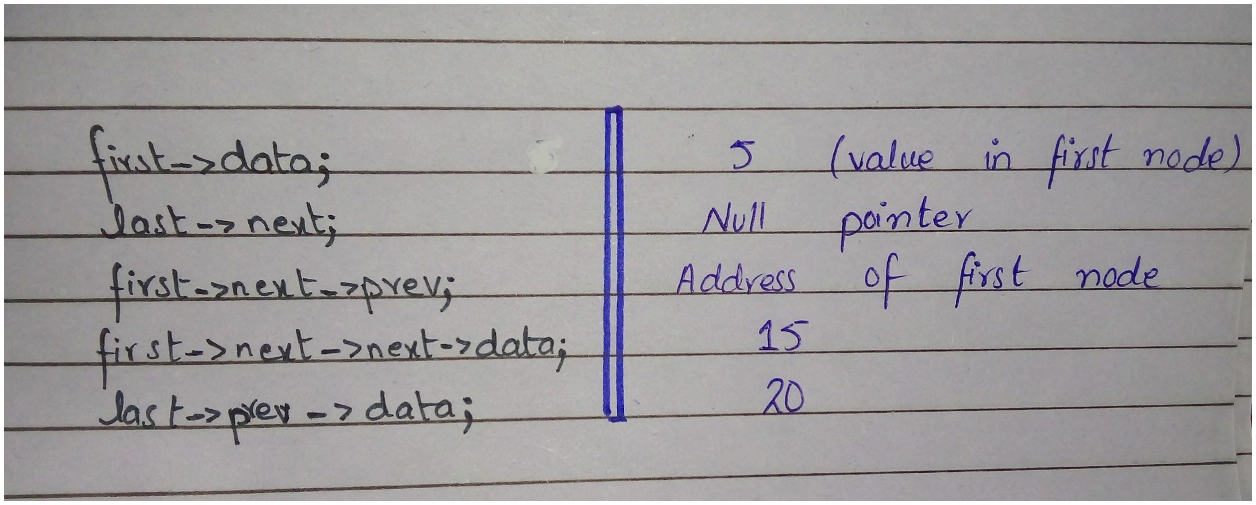
Name : Samreen Zia
Submitted to : Mr. Rehan Ahmed
Semester : BSE (3A)
Date : 01- Dec- 2025

LAB NO 8

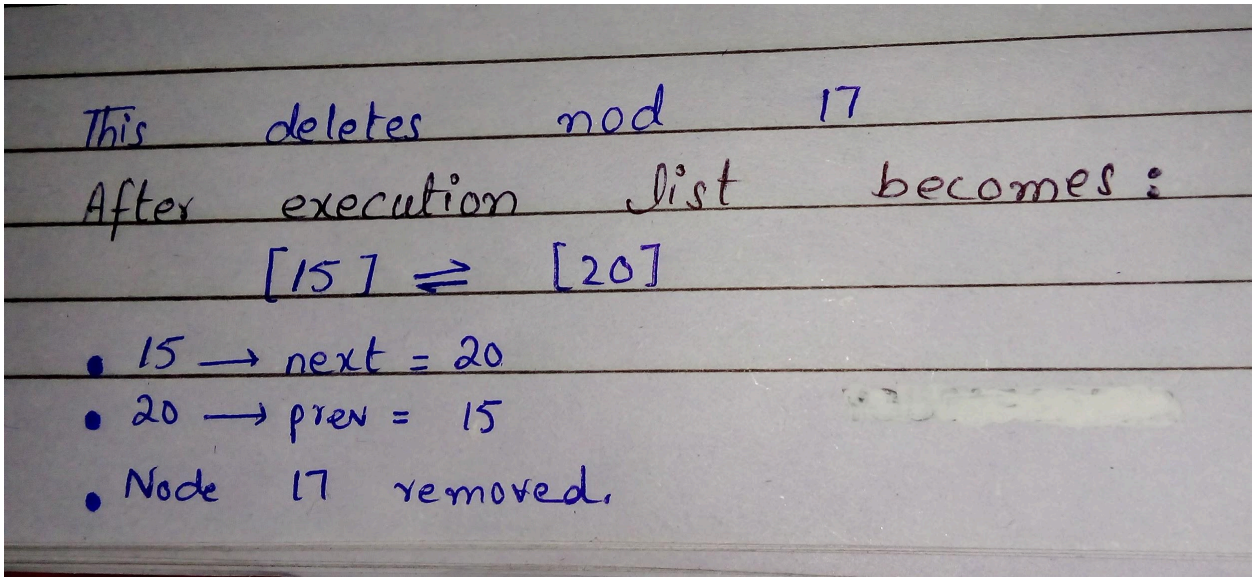
1.
Consider the following doubly linked list.
Write C++ statements to:
a. Print the value 26 using the pointer ‘last’:
b. Print the value 17 using the pointer ‘first:
c. Print the address of the node containing value 9 using the pointer ‘last’:



2.
Given the following linked list, state what does each of the following statements refer t
first->data;
last->next;
first->next->prev;
first->next->next->data;
last->prev->data;



3.
Redraw the following list after the given instructions are executed:
ptr->next->prev = ptr->prev;
ptr->prev->next = ptr->next;
delete ptr;



Task 1:

Implement the class Doubly Linked List, with all provided functions.

```
class DList
{
private:
struct node
{int data;
node *next;
node *prev;
} *head;
public:
DList();
~DList();
bool emptyList();// Checks if the list is empty or not
void insertafter(int oldV, int newV);
// Inserts a new node with value ‘newV’ after the node containing value
‘oldV’. If a node with value ‘oldV’ does not exist, inserts the new node at the end.
void deleteNode(int value);
// Deletes the node containing the specified value
void insert_begin(int value);
// Inserts a new node at the start of the list
void insert_end(int value);
// Inserts a new node at the end of the list
void traverse();
// Displays the values stored in the list
void traverse2();
// Displays the values stored in the list in reverse order
};
```

Program:

```
#include <iostream>
using namespace std;

class DList
{
private:
   struct node {
      int data;
      node *next;
      node *prev;
   } *head;

public:
```

```
DList() {
    head = NULL;
}

~DList() {
    node* temp = head;
    while (temp) {
        node* next = temp->next;
        delete temp;
        temp = next;
    }
}

bool emptyList() {
    return head == NULL;
}

void insert_begin(int value) {
    node* newNode = new node{value, NULL, NULL};
    if (head == NULL) {
        head = newNode;
    } else {
        newNode->next = head;
        head->prev = newNode;
        head = newNode;
    }
}

void insert_end(int value) {
    node* newNode = new node{value, NULL, NULL};
    if (head == NULL) {
        head = newNode;
        return;
    }
    node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
    newNode->prev = temp;
}

void insertafter(int oldV, int newV) {
    node* temp = head;
```

```
    while (temp != NULL && temp->data != oldV)
        temp = temp->next;

    node* newNode = new node{newV, NULL, NULL};

    if (temp == NULL) { // oldV not found → insert at end
        insert_end(newV);
        return;
    }

    newNode->next = temp->next;
    newNode->prev = temp;
    if (temp->next != NULL)
        temp->next->prev = newNode;
    temp->next = newNode;
}

void deleteNode(int value) {
    if (head == NULL) return;

    node* temp = head;

    while (temp != NULL && temp->data != value)
        temp = temp->next;

    if (temp == NULL) return; // value not found

    if (temp->prev) temp->prev->next = temp->next;
    else head = temp->next; // deleting head

    if (temp->next) temp->next->prev = temp->prev;

    delete temp;
}

void traverse() {
    node* temp = head;
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}
```

```
    void traverse2() {
        if (head == NULL) return;

        node* temp = head;
        while (temp->next != NULL)
            temp = temp->next;

        while (temp != NULL) {
            cout << temp->data << " ";
            temp = temp->prev;
        }
        cout << endl;
    }
};
```

Output:

```
Forward traversal: 5 10 15 20
Backward traversal: 20 15 10 5
After deletion: 5 15 20
```

Task 2:

A stack can be implemented using a Doubly linked list. The first node can serve as the 'top' of Stack and 'push' and 'pop' operations can be implemented by adding and removing nodes at the head of the linked list. Implement the Stack class using a linked list (Doubly) and provide all the standard member functions.Data type to store in the stack must be char.

Program:

```
class CharStack {
private:
    struct node {
        char data;
        node* next;
        node* prev;
    } *top;

public:
```

```
CharStack() {
    top = NULL;
}

~CharStack() {
    node* temp = top;
    while (temp) {
        node* next = temp->next;
        delete temp;
        temp = next;
    }
}

bool isEmpty() {
    return top == NULL;
}

void push(char value) {
    node* newNode = new node{value, NULL, NULL};

    if (top == NULL) {
        top = newNode;
    } else {
        newNode->next = top;
        top->prev = newNode;
        top = newNode;
    }
}

char pop() {
    if (isEmpty()) {
        cout << "Stack is empty!\n";
        return '\0';
    }

    char val = top->data;
    node* temp = top;
    top = top->next;
    if (top) top->prev = NULL;
    delete temp;

    return val;
}
```

```
    char peek() {
        if (isEmpty()) {
            cout << "Stack is empty!\n";
            return '\0';
        }
        return top->data;
    }

    void display() {
        node* temp = top;
        while (temp != NULL) {
            cout << temp->data << " ";
            temp = temp->next;
        }
        cout << endl;
    }
};
```

Output: